C++ Programming: From Problem Analysis to Program Design, Third Edition

Chapter 6: User-Defined Functions I

Break STATEMENT

The break statement enables the program to skip over the part of the code. A break statement terminates the smallest enclosing while,do-while for or switch statement,

Break STATEMENT - EXAMPLE

```
while (test expression)
{
        statement
        if(val>2000)
        break;
        …..
        statement;
}
statement 3;
```

Break STATEMENT - EXAMPLE

```
for( int;test expression;update expression)
   {
        statement
        if(val>2000)
        break;
        …..
        statement;
}
statement 3;
```

Break STATEMENT - EXAMPLE

```
do {
    statement
    if(val>2000)
    break;
    …..
    statement;
    } while (test expression);
statement 3;
```

THE continue STATEMENT

- The continue statement is the another jump statement like the break as the both the statements skips over the part of code but the continue statement is some what different than the break. Instead of forcing for termination it forces for the next iteration of the loop to take place.

continue STATEMENT EXAMPLE

```
while (test expression)
   {
        statement
        if(condition)
        continue;
        …..
        statement;
}
statement 3;
```

The continue STATEMENT EXAMPLE

```
for (int;      test expression; updateexpression )
   {
         statement
         if(condition)
         continue;
         …..
         statement;
}
statement 3;
```

The continue STATEMENT EXAMPLE

```
do
  {
        statement
        if(condition)
        continue;
        …..
        statement;
} while (test expression);
statement 3;
```

THE exit() FUNTION

- The exit() function causes the program to terminate as soon as it is encountered.
- The exit() function is a C++ standard library function defined in process.h file.

  which must be included in the program that uses exit() function

EXAMPLE-exit() FUNCTION

```
// Program to check entered number is prime number or not
#include<iostream.h>
#include<conio.h>
#include<process.h>
void main()
{
int num,i;
clrscr();
cout<<"Enter the Number: ";
cin>>num;
for(i=2; i<=num/2;++i)
{
cout<<"\n Not a Prime Number";
exit(0);
}
cout<<"\n It is Prime Number";
getch();
}
```

```
Enter the Number: 4
Not a Prime Number
```

Objectives

In this chapter you will:

- Learn about standard (predefined) functions and discover how to use them in a program
- Learn about user-defined functions
- Examine value-returning functions, including actual and formal parameters
- Explore how to construct and use a value-returning, user-defined function in a program

Functions

- Functions are like building blocks
- They allow complicated programs to be divided into manageable pieces
- Some advantages of functions:
  - A programmer can focus on just that part of the program and construct it, debug it, and perfect it
  - Different people can work on different functions simultaneously
  - Can be used in more than one place in a program or in different programs

Predefined Functions

- In algebra, a function is defined as a rule or correspondence between values, called the function's arguments, and the unique value of the function associated with the arguments
- If $f(x) = 2x + 5$, then $f(1) = 7$, $f(2) = 9$, and $f(3) = 11$
- 1, 2, and 3 are arguments
- 7, 9, and 11 are the corresponding values

Predefined Functions (continued)

- Some of the predefined mathematical functions are:

  `sqrt(x)`

  `pow(x,y)`

  `floor(x)`

- Predefined functions are organized into separate libraries
- I/O functions are in `iostream` header
- Math functions are in `cmath` header

The Power Function (`pow`)

- `pow(x,y)` calculates x^y, `pow(2,3) = 8.0`
- `pow` returns a value of type `double`
- `x` and `y` are called the parameters (or arguments) of the function `pow`
- Function `pow` has two parameters

The `sqrt` and `floor` Functions

- The square root function `sqrt(x)`
  - Calculates the non-negative square root of `x`, for `x >= 0.0`
  - `sqrt(2.25)` is `1.5`
  - Type `double`
  - Has only one parameter

The `sqrt` and `floor` Functions (continued)

- The `floor` function `floor(x)`
  - Returns the largest integer that is smaller than or equal to x (i.e : rounds downs the nearest integer) of `x`
  - `floor(48.79)` is `48.0`
  - Type `double`
  - Has only one parameter

TABLE 6-1 Predefined Functions

Function	Header File	Purpose	Parameter(s) Type	Result
`abs(x)`	`<cstdlib>`	Returns the absolute value of its argument: `abs(-7) = 7`	`int`	`int`
`ceil(x)`	`<cmath>`	Returns the smallest whole number that is not less than `x`: `ceil(56.34) = 57.0`	`double`	`double`
`cos(x)`	`<cmath>`	Returns the cosine of angle `x`: `cos(0.0) = 1.0`	`double` (radians)	`double`
`exp(x)`	`<cmath>`	Returns e^x, where `e = 2.718`: `exp(1.0) = 2.71828`	`double`	`double`
`fabs(x)`	`<cmath>`	Returns the absolute value of its argument: `fabs(-5.67) = 5.67`	`double`	`double`

Ceil Function Returns the integer that is greater than to x (i.e : rounds up the nearest integer)

TABLE 6-1 Predefined Functions (continued)

Function	Header File	Purpose	Parameter(s) Type	Result
`floor(x)`	`<cmath>`	Returns the largest whole number that is not greater than `x`: `floor(45.67) = 45.00`	`double`	`double`
`pow(x, y)`	`<cmath>`	Returns x^y; If `x` is negative, `y` must be a whole number: `pow(0.16, 0.5) = 0.4`	`double`	`double`
`tolower(x)`	`<cctype>`	Returns the lowercase value of `x` if `x` is uppercase; otherwise, returns `x`	`int`	`int`
`toupper(x)`	`<cctype>`	Returns the uppercase value of `x` if `x` is lowercase; otherwise, returns `x`	`int`	`int`

EXAMPLE 6-1

```
//How to use predefined functions.

#include <iostream>
#include <cmath>
#include <cctype>
#include <cstdlib>

using namespace std;

int main()
{
    int   x;
    double u, v;

    cout << "Line 1: Uppercase a is "
         << static_cast<char>(toupper('a'))
         << endl;                                    //Line 1

    u = 4.2;                                         //Line 2
    v = 3.0;                                         //Line 3
    cout << "Line 4: " << u << " to the power of "
         << v << " = " << pow(u, v) << endl;         //Line 4

    cout << "Line 5: 5 to the power of 4 = "
         << pow(5, 4) << endl;                       //Line 5
```

```
    u = u + pow(3, 3);                                  //Line 6
    cout << "Line 7: u = " << u << endl;                //Line 7

    x = -15;                                            //Line 8
    cout << "Line 9: Absolute value of " << x
         << " = " << abs(x) << endl;                    //Line 9

    return 0;
}
```

Sample Run:

```
Line 1: Uppercase a is A
Line 4: 4.2 to the power of 3 = 74.088
Line 5: 5 to the power of 4 = 625
Line 7: u = 31.2
Line 9: Absolute value of -15 = 15
```

User-Defined Functions

- Value-returning functions: have a data type
- Void functions: do not have a return type
- To use these functions you need to:
  - Include the correct header file
  - Know the name of the function
  - Know the number of parameters, if any
  - Know the data type of each parameter
  - Know the data type of the value computed by the function, called the type of the function

Value-Returning Functions

- Because a value-returning function returns value, we can:
  - Save the value for further calculation
  - Use the value in some calculation
  - Print the value
- A value-returning function is used in an assignment or in an output statement

Value-Returning Functions (continued)

- Properties that form the function definition:
  1. Name of the function
  2. Number of parameters
  3. Data type of each parameter
  4. Type of the function
  5. Code required to accomplish the task (the body of the function)

Value-Returning Functions (continued)

- Heading: first four properties above
- Formal/Dummy Parameter: variable declared in the header
- Actual Parameter: variable or expression listed in a call to a function

Value-Returning Functions (continued)

- Syntax:

```
functionType functionName(formal parameter list)
{
    statements
}
```

- `functionType`: type of the value returned by the function
  - Also called the data type
  - Type can be Void if function returns nothing

Syntax

- The syntax of the formal parameter list is:

```
dataType identifier, dataType identifier, ...
```

- The syntax for a function call is:

```
functionName(actual parameter list)
```

- The syntax for the actual parameter list is:

```
expression or variable , expression or variable, ...
```

Functions

- The function parameter list can be empty
- If the formal parameter list is empty
  - Function heading of the value-returning function takes either of the following forms:

    ```
    functionType functionName()
    ```

  - In a function call the actual parameter is empty
- A call to a value-returning function with an empty formal parameter list is:

  ```
  functionName()
  ```

Value-Returning Functions

- To call a value-returning function:
  - Use its name, with the actual parameters (if any) in parentheses
  - There is a one-to-one correspondence between actual and formal parameters

Value-Returning Functions (continued)

- A value-returning function is called in an expression
- Expression may be part of an assignment statement or an output statement
- A function call in a program results in the execution of the body of the called function

The `return` Statement

- Once the function computes the value, the function returns the value via the return statement
- The syntax of the return statement is:

```
return expr;
```

- When a return statement executes
  - Function immediately terminates
  - Control goes back to the caller
- When a return statement executes in the function main, the program terminates

```
double larger(double x, double y)
{
    double max;

    if (x >= y)
        max = x;
    else
        max = y;

    return max;
}
```

You can also write this function as follows:

```
double larger(double x, double y)
{
    if (x >= y)
        return x;
    else
        return y;
}
```

```
double larger(double x, double y)
{
    if (x >= y)
        return x;

    return y;
}
```

NOTE

1. In the definition of the function `larger`, `x` and `y` are formal parameters.
2. The `return` statement can appear anywhere in the function. Recall that once a `return` statement executes, all subsequent statements are skipped. Thus, it's a good idea to return the value as soon as it is computed.

Function Prototype

- Function Prototype: function heading without the body of the function
- Syntax:

```
functionType functionName(parameter list);
```

- It is not necessary to specify the variable name in the parameter list
- The data type of each parameter must be specified

```
//Program: Largest of three numbers

#include <iostream>

using namespace std;

double larger(double x, double y);
double compareThree(double x, double y, double z);

int main()
{
    double one, two;                                    //Line 1

    cout << "Line 2: The larger of 5 and 10 is "
         << larger(5, 10) << endl;                      //Line 2

    cout << "Line 3: Enter two numbers: ";              //Line 3
    cin >> one >> two;                                  //Line 4
    cout << endl;                                       //Line 5

    cout << "Line 6: The larger of " << one
         << " and " << two << " is "
         << larger(one, two) << endl;                   //Line 6
```

```
    cout << "Line 7: The largest of 23, 34, and "
         << "12 is " << compareThree(23, 34, 12)
         << endl;                                    //Line 7

    return 0;
}

double larger(double x, double y)
{
    if (x >= y)
        return x;
    else
        return y;
}

double compareThree (double x, double y, double z)
{
    return larger(x, larger(y, z));
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 2: The larger of 5 and 10 is 10
Line 3: Enter two numbers: 25 73

Line 6: The larger of 25 and 73 is 73
Line 7: The largest of 23, 34, and 12 is 34
```

Flow of Execution

- Execution always begins at
  - The first statement in the function `main` no matter where `main` is placed in the program
- Other functions are executed only when they are called

Flow of Execution (Continued)

- Function prototypes appear before any function definition
  - The compiler translates these first
- The compiler can then correctly translate a function call

Flow of Execution (continued)

- A function call statement results in
  - Transfer of control to the first statement in the body of the called function
- After the last statement of the called function is executed
  - Control is passed back to the point immediately following the function call

Flow of Execution (continued)

- A value-returning function returns a value
- After executing the function
  - The value that the function returns replaces the function call statement

Programming Example

- In this programming example, the function `larger` is used to determine the largest number from a set of numbers
- Program determines the largest number from a set of 10 numbers
- Input: A set of 10 numbers
- Output: The largest of 10 numbers

Program Analysis

- Suppose that the input data is:

  `15 20 7 8 28 21 43 12 35 3`

- Read the first number of the data set
- Because this is the only number read to this point, you may assume that it is the largest number so far and call it `max`
- Read the second number and call it `num`
- Compare `max` and `num`, and store the larger number into `max`

Program Analysis (continued)

- Now `max` contains the larger of the first two numbers
- Read the third number and compare it with `max` and store the larger number into `max`
- At this point, `max` contains the largest of the first three numbers
- Read the next number, compare it with `max`, and store the larger into `max`
- Repeat this process for each remaining number in the data set

Algorithm Design

- Read the first number
  - Because this is the only number that you have read, it is the largest number so far
  - Save it in a variable called `max`
- For each remaining number in the list
  - Read the next number
  - Store it in a variable called `num`
  - Compare `num` and `max`

Algorithm Design (continued)

- If `max < num`
  - `num` is the new largest number
  - update the value of `max` by copying `num` into `max`
- If `max >= num`, discard `num`; that is, do nothing

- Because `max` now contains the largest number, print it